

Report

Exercise 2 - MySQL

Group: 82

Student: Fredrik August Flekstad Lavik

Introduction

I completed the exercise individually using the Porto Taxi Trajectory dataset. The work involved exploratory data analysis, data cleaning, schema design, data import, and analytical queries using Python and MySQL. MySQL ran locally in Docker, and Git was used for version control.

Repository: <https://github.com/verinix/tdt4225-exercise-2>

Results

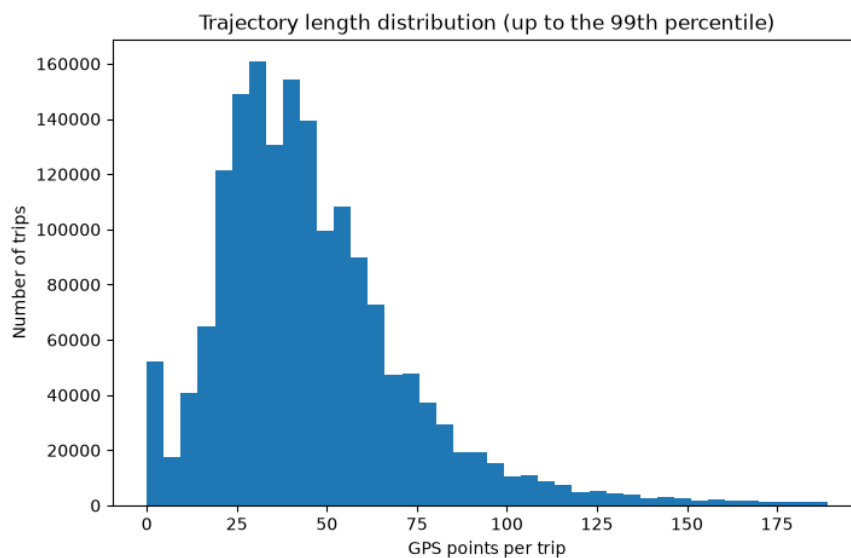
Part 1 - EDA, data cleaning, and schema design

I began by inspecting the main distributions and data-quality issues before deciding how to clean and store the data.

Raw dataset	Result
Trips	1,710,670
Distinct taxis	448
GPS observations	83,409,386
Empty trajectories	5,901
Invalid trips	43,904
Duplicated trip IDs	80
Identical trip IDs	2
Conflicting trip IDs	78

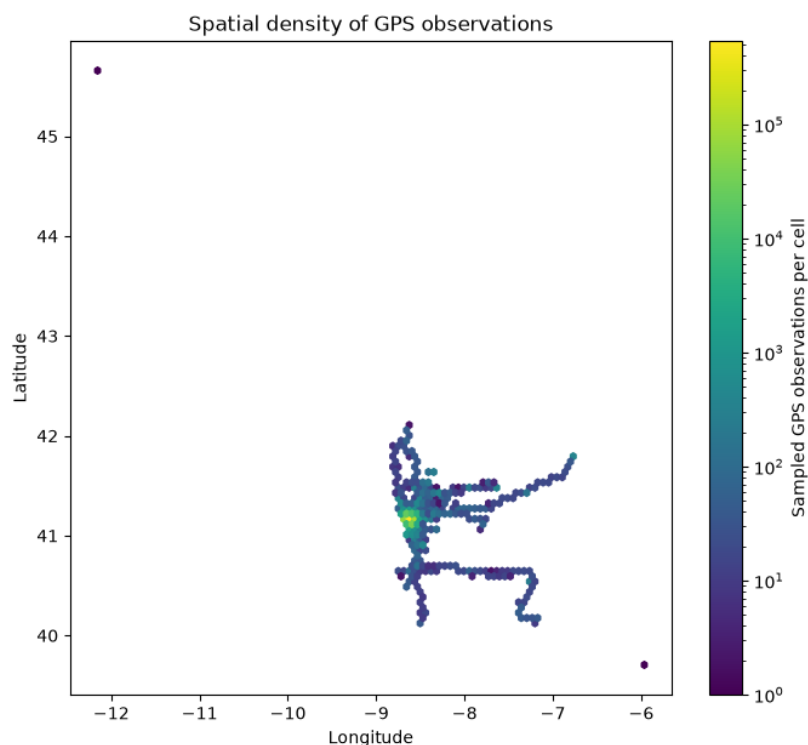
All three call types were present, with B being the most common, while DAY_TYPE was A for every trip.

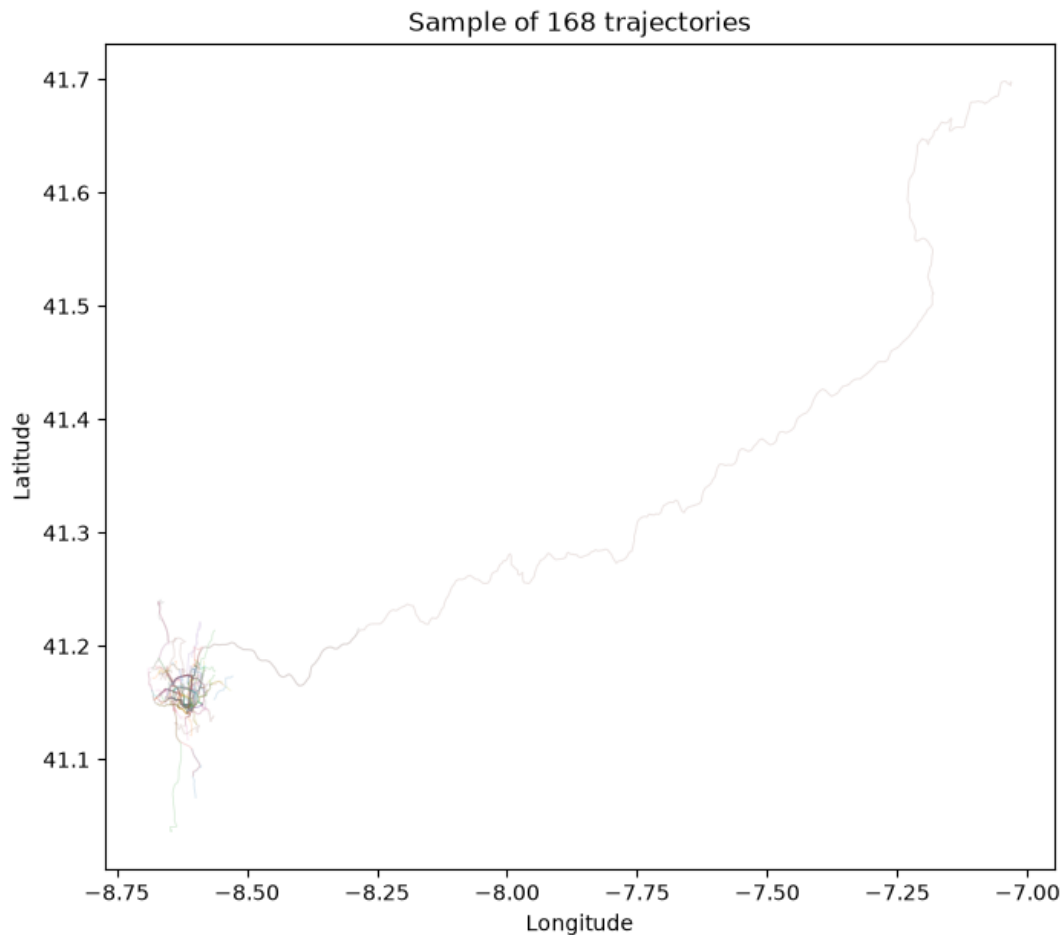
Trips contained an average of 48.76 GPS points and a median of 41. The 95th and 99th percentiles were 103 and 189 points, while the longest trajectory contained 3,881 points.



The supplied MISSING_DATA flag and the exercise definition of an invalid trip described different problems: only 10 rows had MISSING_DATA=True, while 43,904 trips contained fewer than three GPS points. I therefore kept the two cases separate.

The spatial plots were centered on Porto but also showed observations far outside the main cluster. Since a distant trip is not necessarily invalid, I did not remove trips based on location alone.





This caused me to inspect consecutive GPS observations. I found clearly implausible jumps, including hundreds of kilometers between 15-second samples. I preserved the original points and stored the maximum consecutive jump for each trip so distance-sensitive queries could later exclude unreliable trajectories.

Cleaning

I kept one copy of identical duplicates and removed all records for conflicting TRIP_ID values because there was no reliable way to choose between them. I kept invalid trips and the MISSING_DATA flag so later queries could distinguish them. Empty origin fields were stored as NULL.

Schema

I used two tables: trip for trip metadata and trajectory_point for the ordered GPS observations. I did not create a separate taxi table because the dataset contained no taxi attributes beyond TAXI_ID.

```

CREATE TABLE trip (
    trip_id BIGINT UNSIGNED NOT NULL,
    taxi_id INT UNSIGNED NOT NULL,
    call_type CHAR(1) NOT NULL,
    origin_call INT UNSIGNED NULL,
    origin_stand INT UNSIGNED NULL,
    start_time DATETIME NOT NULL,
    day_type CHAR(1) NOT NULL,
    missing_data BOOLEAN NOT NULL,
    point_count SMALLINT UNSIGNED NOT NULL,
    distance_km DOUBLE NOT NULL,
    max_jump_km DOUBLE NULL,

    PRIMARY KEY (trip_id),
    CHECK (call_type IN ('A', 'B', 'C')),
    CHECK (day_type IN ('A', 'B', 'C'))
)

CREATE TABLE trajectory_point (
    trip_id BIGINT UNSIGNED NOT NULL,
    point_index SMALLINT UNSIGNED NOT NULL,
    longitude DOUBLE NOT NULL,
    latitude DOUBLE NOT NULL,

    PRIMARY KEY (trip_id, point_index),
    FOREIGN KEY (trip_id) REFERENCES trip(trip_id)
)

CREATE INDEX idx_trip_taxi_start ON trip(taxi_id, start_time)

```

While designing the schema, I looked ahead at the analytical tasks. `point_count`, `distance_km`, and `max_jump_km` were inexpensive to calculate during import and likely to be reused, so I stored them with each trip instead of recalculating them from more than 83 million rows. I also added a secondary index on `(taxi_id, start_time)`, since several tasks concerned taxis and chronological trip order.

Import result

Database check	Result
Rows in trip	1,710,511
Rows in trajectory_point	83,407,339
SUM(trip.point_count)	83,407,339

After cleaning, the database contained 1,710,511 trips and 83,407,339 trajectory points. The trajectory_point count matched SUM(trip.point_count), suggesting that import succeeded.

Part 2 - Analytical queries

For duration and distance calculations, I excluded invalid or incomplete trajectories. Distance-sensitive calculations also excluded trips with consecutive jumps greater than 1 km. I chose 1 km because a 0.5 km threshold would exclude about 15% of trips, while 1 km excluded only about 1.2% and corresponded to roughly 240 km/h between 15-second observations, which I considered to be implausible for any taxi.

Tasks 1-3

The cleaned database contained 448 taxis, 1,710,511 trips, and 83,407,339 GPS observations, with an average of 3,818.10 trips per taxi. The top 20 taxis by trip count are shown below.

```

Task 1
Measure      Count
-----
Taxis        448
Trips        1710511
GPS points   83407339

Task 2
Average trips per taxi: 3818.10

Task 3
taxi_id      trip_count
-----
20000080     10716
20000403      9237
20000066      8437
20000364      7821
20000483      7729
20000129      7607
20000307      7496
20000621      7276
20000089      7265
20000424      7176
20000492      7171
20000529      6937
20000616      6924
20000678      6538
20000372      6535
20000304      6505
20000042      6467
20000325      6460
20000179      6430
20000235      6410

```

Task 4

Below is the most common call type for each taxi together with average duration, average distance, and the distribution of trip starts across the four six-hour periods. Ties for the most common call type were kept.

Trip duration was estimated from the 15-second sampling interval. The averages use valid, complete trajectories without consecutive jumps above 1 km.

Task 4a

taxi_id	call_type	trip_count
20000001	B	1311
20000002	B	1576
20000003	C	844
20000004	B	2680
20000005	B	3034
20000006	B	1702
20000007	B	2770
20000008	B	2665
20000009	B	2099
20000010	B	2842
20000011	B	2622
20000012	B	2257
20000013	B	2581
20000014	B	2523
20000015	B	2819
20000017	B	1129
20000018	B	958
20000020	B	2304
20000021	B	1901
20000022	B	1990

Showing first 20 taxis.

Task 4b

call_type	avg_duration_min	avg_distance_km	00-06 %	06-12 %	12-18 %	18-24 %
A	12.57	5.36	11.93	33.35	32.17	22.55
B	11.20	5.02	12.23	30.12	34.46	23.18
C	13.49	6.43	32.26	23.39	24.39	19.96

Task 5

The top taxis by total driving time are shown below together with their total distance. The same reliable trajectory subset was used for both measures.

Task 5

taxi_id	total_hours	total_distance_km
20000904	1882.46	59117.33
20000129	1636.50	35943.82
20000307	1568.88	39076.09
20000529	1485.13	41422.91
20000436	1388.22	43085.22
20000483	1386.17	35613.73
20000372	1372.67	39844.34
20000276	1334.37	42037.88
20000616	1320.64	34475.67
20000179	1306.59	38804.59
20000621	1274.61	31595.76
20000574	1270.78	35831.38
20000364	1270.55	40980.88
20000235	1265.71	35352.84
20000011	1261.63	38155.34
20000395	1248.83	33594.83
20000492	1246.97	32390.87
20000199	1242.57	37249.36
20000042	1237.72	31892.71
20000435	1232.98	38249.48

Showing first 20 taxis.

Task 6-9

I found 126,532 trips that passed within 100 m of Porto City Hall and 43,800 invalid trips. There were 8,074 midnight crossers and 21,460 circular trips whose start and end points were within 50 m of each other.

```
Task 6: trips within 100 m of Porto City Hall
Trips found: 126,532
      trip_id
-----
1372636896620000360
1372636956620000167
1372637423620000341
1372637610620000497
1372637721620000678
1372638361620000154
1372638851620000406
1372639364620000015
1372639441620000012
1372640338620000423
1372641197620000653
1372641991620000231
1372642071620000305
1372642706620000653
1372643415620000653
1372643825620000569
1372646982620000600
1372648688620000308
1372649313620000231
1372650736620000012
Showing first 20 trips.
```

```
Task 7
Invalid trips: 43,800
```

```
Task 8: midnight crossers
Trips found: 8,074
      trip_id
-----
1372722134620000435
1372722457620000569
1372722471620000540
1372722555620000680
1372722610620000297
1372722646620000351
1372722663620000108
1372722796620000451
1372722801620000015
1372722877620000598
1372722880620000005
1372722892620000307
1372722919620000603
1372722923620000273
1372722970620000195
1372722985620000454
1372723001620000483
1372723054620000014
1372723060620000337
1372723069620000688
Showing first 20 trips.
```

```
Task 9: circular trips
Trips found: 21,460
      trip_id
-----
1372638303620000112
1372639092620000233
1372644642620000305
1372650711620000403
1372650908620000403
1372661911620000496
1372662261620000031
1372663272620000206
1372664679620000186
1372664787620000397
1372665027620000574
1372667532620000443
1372667871620000152
1372667893620000116
1372668421620000093
1372669341620000388
1372669439620000460
1372669656620000184
1372669944620000398
1372669996620000616
Showing first 20 trips.
```

Task 10

I treated idle time as the gap between the estimated end of one trip and the start of the next trip for the same taxi. The top 20 taxis by average idle time are shown below.

```
Task 10
      taxi_id      avg_idle_hours
-----
20000941          114.36
20000969           54.24
20000312           13.08
20000510           12.34
20000609           11.06
20000579           11.05
20000079            9.00
20000185            8.94
20000449            8.67
20000902            7.44
20000072            7.08
20000535            6.61
20000315            6.36
20000071            6.24
20000225            5.95
20000545            5.89
20000407            5.63
20000443            5.40
20000205            5.31
20000634            5.23
```

Discussion

The main cleaning decision concerned duplicate TRIP_ID values. Identical duplicates were reduced to one copy, while conflicting duplicates were removed because there was no reliable way to choose between them. The timestamps were often identical, and choosing the first, longest, or otherwise preferred trajectory would have been arbitrary. Invalid and incomplete trajectories were kept because they represented separate data-quality cases and were still useful in later analysis.

The EDA also showed that location alone was not a good cleaning rule. Some long trips appeared plausible, while impossible jumps between consecutive 15-second observations directly distorted distance calculations. I therefore preserved the original observations and stored max_jump_km, which allowed the 1 km quality threshold to be applied only when distance mattered.

The main practical challenge was the data volume. I processed the CSV incrementally and used batched inserts, since flattening the trajectories produced more than 83 million rows. The derived trip values avoided repeatedly recalculating information from this large table, while the (taxi_id, start_time) index supported the many taxi-based queries and chronological trip processing. The City Hall proximity query remained the most expensive because it had to scan the large trajectory_point table.

I computed the complete result sets, but only the first 20 rows are displayed to keep screenshots readable.

Feedback

No specific feedback.

AI use

I used AI to assist with writing code for spatial plots and percentiles. I also used AI to review the submission for correctness.